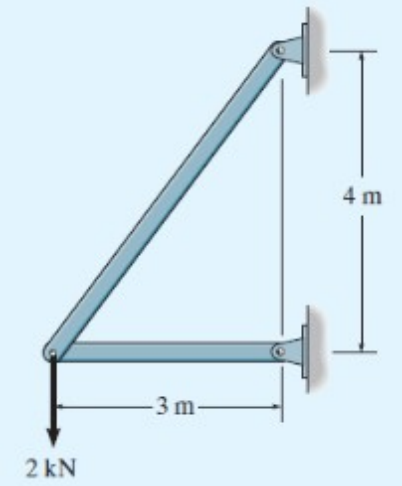
Exercise 1 — Plane truss analysis step by step

This Live Script explains how the direct stiffness method converts a structural model into joint displacements, support reactions and axial member forces. Run the sections **from top to bottom**, using **Run Section**, and inspect the displayed intermediate results before continuing. Later sections depend on variables created earlier.

Exercise: a 2 kN downward load acts at node 2. Member 1 joins nodes 2 and 3 horizontally; member 2 joins nodes 2 and 1 diagonally. Nodes 1 and 3 are restrained in both directions. The horizontal span is 3 m and the height is 4 m.



Assumptions: straight bars, ideal pinned joints, axial deformation only, linear elasticity, small displacements and zero support settlements. Use N, m and Pa internally. Positive axial force means tension; negative axial force means compression.

No results are precomputed or embedded in this file. Run it to generate the tables and figures. The code has been reviewed and its numerical results independently checked, but this file has not been executed or rendered in MATLAB in the preparation environment.

1 Start with a clean workspace

`clear` removes variables from the workspace, `clc` clears the Command Window and `close all` closes open figures. These commands prepare a fresh run; they are not part of the stiffness formulation. Run this section once at the start, not between later sections.

```
clear;
clc;
close all;
```

2 Describe the joints with coordinates

Each row of `nodes` represents one joint. Column 1 contains x and column 2 contains y, in metres. Thus `nodes(2,:)` returns both coordinates of node 2, while `nodes(1,2)` returns only the y coordinate of node 1. A colon means all entries along that dimension. Spaces separate columns; semicolons separate rows.

```
nodes = [3 4;       % Node 1: upper wall joint
```

```
          0 0;       % Node 2: loaded joint
          3 0];      % Node 3: lower wall joint
```

`size(nodes,1)` counts rows, so it gives the number of nodes. The semicolon at the end of an assignment suppresses automatic output. Here we deliberately display the coordinates as a labelled table using `disp` and `table`.

```
nNodes = size(nodes,1);
disp(table((1:nNodes).',nodes(:,1),nodes(:,2), ...
    'VariableNames',{'Node','x_m','y_m'}));
```

Node	x_m	y_m
1	3	4
2	0	0
3	3	0

`(1:nNodes)` creates the sequence 1, 2, 3. The operator `.'` transposes this row into a column so it fits a table column. The `...` symbol continues one MATLAB statement onto the next line; it does not change the calculation.

3 Describe which nodes each member connects

Each row of `members` stores two **node numbers**, not coordinates. Row 1 defines member 1 from node 2 to node 3. Row 2 defines member 2 from node 2 to node 1. The selected start and end directions must be used consistently when forming stiffness and recovering forces.

```
members = [2 3;     % Member 1: horizontal
           2 1];    % Member 2: inclined
nMembers = size(members,1);
disp(table((1:nMembers).',members(:,1),members(:,2), ...
    'VariableNames',{'Member','StartNode','EndNode'}));
```

Member	StartNode	EndNode
1	2	3
2	2	1

4 Define material properties and units

A bar resists axial extension through its stiffness EA/L. `E` is Young's modulus in Pa, `A` is cross-sectional area in square metres and `L` will be calculated in metres. `200e9` means 200 times 10 to the power 9, equivalent to 200 GPa.

The supplied first exercise states that EA is constant but does not specify separate numerical E and A values. We adopt the following teaching values to calculate displacements. The same E and A are used for both members.

```
E = 200e9;          % Young's modulus, Pa
A = 0.0015;         % Cross-sectional area, m^2
```

5 Number the degrees of freedom

Each joint can move horizontally and vertically, so it has two degrees of freedom (DOFs). Use the rule **node i: horizontal DOF = 2i - 1; vertical DOF = 2i**. The displacement vector is therefore ordered as `[ux1; uy1; ux2; uy2; ux3; uy3]`.

This ordering differs from the textbook's displacement arrow numbers. Keep its node labels but always use the MATLAB DOF rule below. The loaded joint is node 2, so its horizontal and vertical displacements are `U(3)` and `U(4)`.

```
nDOF = 2*nNodes;
nodeIDs = (1:nNodes).';
xDOFs = 2*nodeIDs-1;
yDOFs = 2*nodeIDs;
disp(table(nodeIDs,xDOFs,yDOFs, ...
    'VariableNames',{'Node','HorizontalDOF','VerticalDOF'}));
```

Node	HorizontalDOF	VerticalDOF
1	1	2
2	3	4
3	5	6

6 Apply restraints and external loads

Nodes 1 and 3 cannot move in either direction. Their DOFs are 1, 2, 5 and 6. `setdiff` returns the numbers in `1:nDOF` that do not appear in `fixedDOFs`; these are the unknown displacement components.

```
fixedDOFs = [1 2 5 6];
freeDOFs = setdiff(1:nDOF,fixedDOFs);
disp('Free DOFs:');
```

```
Free DOFs:
```

```
disp(freeDOFs);
```

```
     3     4
```

`zeros(nDOF,1)` creates a column vector with six zero entries. Set only the loaded entry of F. A positive vertical load acts upwards, so the 2 kN downward load is -2000 N at DOF 4. U starts at zero because its restrained entries are prescribed as zero; we will overwrite its free entries after solving.

```
loadDOF = 4;
loadValue = -2000;
F = zeros(nDOF,1);
F(loadDOF) = loadValue;
U = zeros(nDOF,1);
disp(table((1:nDOF).',F, ...
    'VariableNames',{'DOF','AppliedForce_N'}));
```

DOF	AppliedForce_N
1	0

```
     2            0
     3            0
     4        -2000
     5            0
     6            0
```

Checkpoint: `freeDOFs` must be `[3 4]`. The only nonzero entry of F must be F(4) = -2000. The supports are not modelled by setting forces to zero: they are modelled by prescribing displacements.

7 Inspect one inclined member before writing the loop

We first examine member 2 separately so that the geometry and element matrix are visible. `members(eExample,1)` selects its start node and `members(eExample,2)` selects its end node. Subtract the start coordinates from the end coordinates.

```
eExample = 2;
iExample = members(eExample,1);
jExample = members(eExample,2);
dxExample = nodes(jExample,1)-nodes(iExample,1);
dyExample = nodes(jExample,2)-nodes(iExample,2);
LExample = hypot(dxExample,dyExample);
cExample = dxExample/LExample;
sExample = dyExample/LExample;
disp(table(eExample,LExample,cExample,sExample, ...
    'VariableNames',{'Member','Length_m','c','s'}));
```

Member	Length_m	c	s
2	5	0.6	0.8

`hypot(dx,dy)` computes the square root of dx squared plus dy squared. The ratios c = dx/L and s = dy/L are the direction cosines. For member 2, L = 5 m, c = 0.6 and s = 0.8. Keep their signs; another member direction may give negative direction cosines.

The row vector b projects the end displacements onto the member axis. Its product with the four element displacement components gives axial extension: delta = b ue. Axial force is N = (EA/L) delta. The associated global-coordinate element stiffness is ke = (EA/L) b-transpose b.

```
bExample = [-cExample -sExample cExample sExample];
keExample = (E*A/LExample)*(bExample.'*bExample);
disp('Inclined member stiffness matrix ke, in N/m:');
```

```
Inclined member stiffness matrix ke, in N/m:
```

```
disp(keExample);
```

```
   1.0e+07 *

    2.1600    2.8800   -2.1600   -2.8800
    2.8800    3.8400   -2.8800   -3.8400
   -2.1600   -2.8800    2.1600    2.8800
   -2.8800   -3.8400    2.8800    3.8400
```

`bExample` has size 1 by 4; `bExample.'` has size 4 by 1. Their outer product is 4 by 4. Its rows and columns correspond to `[uxi, uyi, uxj, uyj]`. Use ordinary matrix multiplication `*`, not elementwise multiplication `.*`, for this outer product.

Think before continuing: a horizontal member has s = 0. Which rows and columns of its element matrix are zero? It provides axial stiffness in x but no first-order transverse stiffness in y.

8 Assemble all members into the global stiffness matrix

Initialise K as a square zero matrix with one row and column per global DOF. A `for` loop repeats the same operations for each member. The loop variable e takes the values 1 and 2 here. All statements between `for` and its matching `end` belong to the loop.

`dofs` maps the four element displacement positions into the full structure. Member 1 uses `[3 4 5 6]`; member 2 uses `[3 4 1 2]`. `K(dofs,dofs)` selects the corresponding 4 by 4 block, even when its indices are not consecutive.

```
K = zeros(nDOF,nDOF);
for e = 1:nMembers
    i = members(e,1);               % Start node of member e
    j = members(e,2);               % End node of member e
    dx = nodes(j,1)-nodes(i,1);      % Horizontal coordinate difference
    dy = nodes(j,2)-nodes(i,2);      % Vertical coordinate difference
    L = hypot(dx,dy);               % Member length
    c = dx/L;                      % Horizontal direction cosine
    s = dy/L;                      % Vertical direction cosine
    b = [-c -s c s];               % Axial extension projection
    ke = (E*A/L)*(b.'*b);           % Global-coordinate element stiffness
    dofs = [2*i-1 2*i 2*j-1 2*j]; % Element-to-global DOF mapping
    K(dofs,dofs) = K(dofs,dofs) + ke;
end
```

The final line inside the loop **adds** the element stiffness to contributions already stored in K. Replacing it with `K(dofs,dofs) = ke` would erase contributions at shared DOFs. Assembly encodes the fact that connected members share the same nodal displacement.

```
disp('Assembled global stiffness matrix K, in N/m:');
```

```
Assembled global stiffness matrix K, in N/m:
```

```
disp(K);
```

```
   1.0e+08 *

    0.2160    0.2880   -0.2160   -0.2880         0         0
    0.2880    0.3840   -0.2880   -0.3840         0         0
   -0.2160   -0.2880    1.2160    0.2880   -1.0000         0
   -0.2880   -0.3840    0.2880    0.3840         0         0
         0         0   -1.0000         0    1.0000         0
         0         0         0         0         0         0
```

Checkpoint: K must be 6 by 6 and symmetric. Before support conditions are imposed, K is singular because the unconstrained structure has rigid-body motions. That alone does not mean the supported truss is unstable.

9 Reduce the system using known support displacements

At free DOFs, equilibrium gives Kff Uf + Kfc Uc = Ff. The subscript f means free and c means constrained. Since every prescribed displacement Uc is zero in this exercise, the reduced equation is Kff Uf = Ff. Nonzero support settlements would require the additional term -Kfc Uc on the right-hand side.

```
Kff = K(freeDOFs,freeDOFs);
Ff = F(freeDOFs);
disp('Reduced stiffness Kff, in N/m:');
```

```
Reduced stiffness Kff, in N/m:
```

```
disp(Kff);
```

```
   1.0e+08 *

    1.2160    0.2880
    0.2880    0.3840
```

```
disp('Reduced stiffness divided by EA, in 1/m:');
```

```
Reduced stiffness divided by EA, in 1/m:
```

```
disp(Kff/(E*A));
```

```
    0.4053    0.0960
    0.0960    0.1280
```

Expected: Kff/(EA) is approximately `[0.4053333333 0.096; 0.096 0.128]`. It is a 2 by 2 system because only the two displacement components at node 2 are unknown. Use the coordinate-based coefficients rather than the rounded entries printed in the textbook.

`rcond` estimates the reciprocal condition number. A value near zero warns that the reduced system may be singular or poorly conditioned. The threshold below is a diagnostic screen, not a universal proof of model validity.

```
if rcond(Kff) < 1e-12
    error('Unstable or poorly conditioned model: check supports/connectivity.');
end
```

10 Solve the unknown displacements

MATLAB backslash solves a system of simultaneous equations. It is preferable to explicitly calculating an inverse with `inv`. The left side below stores the solution only at free positions in U; the prescribed support entries remain zero.

```
U(freeDOFs) = Kff \ Ff;
```

Display the complete vector in millimetres by multiplying metres by 1000. The colon expression `1:2:end` selects odd positions, which are horizontal displacements. `2:2:end` selects even positions, which are vertical displacements. `end` means the last array index.

```
displacementTable = table((1:nNodes).', ...
    U(1:2:end)*1e3,U(2:2:end)*1e3, ...
    'VariableNames',{'Node','ux_mm','uy_mm'});
disp(displacementTable);
```

Node	ux_mm	uy_mm
1	0	0
2	0.015	-0.063333
3	0	0

Interpretation: node 2 should move +0.015000 mm to the right and -0.063333 mm vertically, meaning downwards. All restrained entries must remain zero. Force equilibrium alone would not provide these displacements; the stiffness and compatibility relations are essential.

11 Recover support reactions

The vector K U is the nodal force vector required to maintain the computed displacement field. Subtract the externally applied nodal load F to obtain reactions at restrained DOFs and numerical residuals at free DOFs. This subtraction is necessary even when loads act at supports.

```
R = K*U-F;
reactionTable = table(fixedDOFs.',R(fixedDOFs)/1e3, ...
    'VariableNames',{'Restrained_DOF','Reaction_kN'});
disp(reactionTable);
```

Restrained_DOF	Reaction_kN
1	1.5
2	2
5	-1.5
6	0

Dividing N by 1000 converts forces to kN. DOFs 1 and 2 are the x and y reactions at node 1: +1.5 and +2.0 kN. DOFs 5 and 6 are the reactions at node 3: -1.5 and 0 kN. The entries R(3) and R(4) should be approximately zero.

12 Recover member extension and axial force

For each member, extract its four nodal displacements with `U(dofs)`. The multiplication `[-c -s c s]*U(dofs)` gives the signed axial extension in metres. Multiplying by EA/L gives axial force in newtons. Positive extension produces tension; negative extension produces compression.

```
N = zeros(nMembers,1);
extension = zeros(nMembers,1);
for e = 1:nMembers
    i = members(e,1);
```

```
    j = members(e,2);
    dx = nodes(j,1)-nodes(i,1);
    dy = nodes(j,2)-nodes(i,2);
    L = hypot(dx,dy);
    c = dx/L;
    s = dy/L;
    dofs = [2*i-1 2*i 2*j-1 2*j];
    extension(e) = [-c -s c s]*U(dofs);
    N(e) = (E*A/L)*extension(e);
end
stress = N/A;                   % Normal stress, Pa
```

Normal stress is axial force divided by area. This model uses one common area for every member. If member areas become a vector, use the area for each member in both stiffness and recovery, and use elementwise division `N./A` when forming stresses.

`repmat` creates a cell array of labels. A logical comparison such as `N > 1e-6` identifies entries in tension; using that comparison as an index updates only those labels. The small threshold prevents floating-point round-off near zero from being reported as a meaningful force.

```
state = repmat({'zero'},nMembers,1);
state(N>1e-6) = {'tension'};
state(N<-1e-6) = {'compression'};
memberTable = table((1:nMembers).',extension*1e3, ...
    N/1e3,stress/1e6,state, ...
    'VariableNames',{'Member','Extension_mm','N_kN','Stress_MPa','State'});
disp(memberTable);
```

Member	Extension_mm	N_kN	Stress_MPa	State
1	-0.015	-1.5	-1	{'compression'}
2	0.041667	2.5	1.6667	{'tension' }

Expected: member 1 carries -1.5 kN in compression and member 2 carries +2.5 kN in tension. The corresponding stresses are -1.000 MPa and +1.666667 MPa. Compression here is an axial force result; it is not a buckling safety check.

13 Print the requested joint displacement

`fprintf` prints formatted text. `%.6f` inserts a number with six digits after the decimal point; `\n` starts a new line. DOF 4 is the vertical displacement at node 2 under the adopted ordering.

```
fprintf('Vertical displacement at node 2 = %.6f mm\n',U(4)*1e3);
```

```
Vertical displacement at node 2 = -0.063333 mm
```

14 Verify symmetry and equilibrium

A numerical answer should satisfy physical checks. `assert` stops with an explanatory message if its condition is false. A tolerance allows for floating-point round-off; it does not replace engineering judgement.

K should equal its transpose. The Frobenius norm measures the size of all matrix entries together. The relative comparison below accounts for the scale of the stiffness entries. It is a matrix symmetry check, not a check of the support arrangement.

```
assert(norm(K-K.','fro') < 1e-10*max(1,norm(K,'fro')), ...
    'The assembled stiffness matrix is not symmetric.');
```

At free DOFs, the largest residual force should be very small. `norm(vector,inf)` returns the largest absolute component.

```
assert(norm(R(freeDOFs),inf) < 1e-6, ...
    'The free DOF force residual is too large.');
```

For the whole structure, add reactions and applied loads. Horizontal and vertical forces must separately sum to zero. A nodal force at position (x,y) contributes the moment x Fy - y Fx about the origin. `.*` multiplies corresponding vector entries, unlike the matrix multiplication used in the solver.

```
net = R+F;
assert(abs(sum(net(1:2:end))) < 1e-6, ...
    'Horizontal force equilibrium failed.');
assert(abs(sum(net(2:2:end))) < 1e-6, ...
    'Vertical force equilibrium failed.');
M = sum(nodes(:,1).*net(2:2:end)-nodes(:,2).*net(1:2:end));
assert(abs(M) < 1e-6,'Moment equilibrium failed.');
fprintf('Symmetry and equilibrium checks passed.\n');
```

```
Symmetry and equilibrium checks passed.
```

Finally compare against the original exercise. The reference checks below run only for the original geometry, properties, supports and load. If you change the model for exploration, equilibrium checks still run but the fixed reference values are skipped.

```
isReference = isequal(nodes,[3 4;0 0;3 0]) && ...
    isequal(members,[2 3;2 1]) && ...
    isequal(fixedDOFs,[1 2 5 6]) && ...
    isequal(F,[0;0;0;-2000;0;0]) && E==200e9 && A==0.0015;
if isReference
    assert(max(abs(N-[-1500;2500])) < 1e-5, ...
        'Member forces do not match the reference exercise.');
    assert(abs(U(3)-0.015e-3) < 1e-12, ...
        'Horizontal displacement does not match the reference.');
    assert(abs(U(4)+0.0633333333333333e-3) < 1e-12, ...
        'Vertical displacement does not match the reference.');
    fprintf('Original exercise reference checks passed.\n');
else
    fprintf('Modified model: original reference checks skipped.\n');
end
```

```
Original exercise reference checks passed.
```

15 Prepare a visible deformed shape

Actual displacements are too small to see beside metre-scale geometry. `reshape(U,2,[]).'` groups successive x and y entries into one row per node. The empty dimension `[]` tells MATLAB to infer the required number of columns before transposing. The resulting matrix has the same size and ordering as `nodes`.

```
Umatrix = reshape(U,2,[]).';
```

`max(nodes)-min(nodes)` gives the coordinate ranges in x and y. The outer `max` selects the larger range as a characteristic span. Squaring and adding each row of Umatrix gives squared displacement magnitude at each node; `sum(...,2)` sums across columns.

```
span = max(max(nodes)-min(nodes));
maxDisp = max(sqrt(sum(Umatrix.^2,2)));
```

Choose a magnification that makes the largest plotted displacement equal to 12 percent of the characteristic span. The `if` statement avoids division by zero if every displacement is zero. This is only a drawing scale; it does not alter U, forces or stresses.

```
scale = 1;
if maxDisp>0
    scale = 0.12*span/maxDisp;
end
moved = nodes + scale*Umatrix;
```

16 Draw and label the structure

`figure` creates a plot with a white background. `hold on` retains earlier lines while new members are drawn. `grid on` adds reference grid lines, and `axis equal` gives the same plotted length to one metre in x and one metre in y.

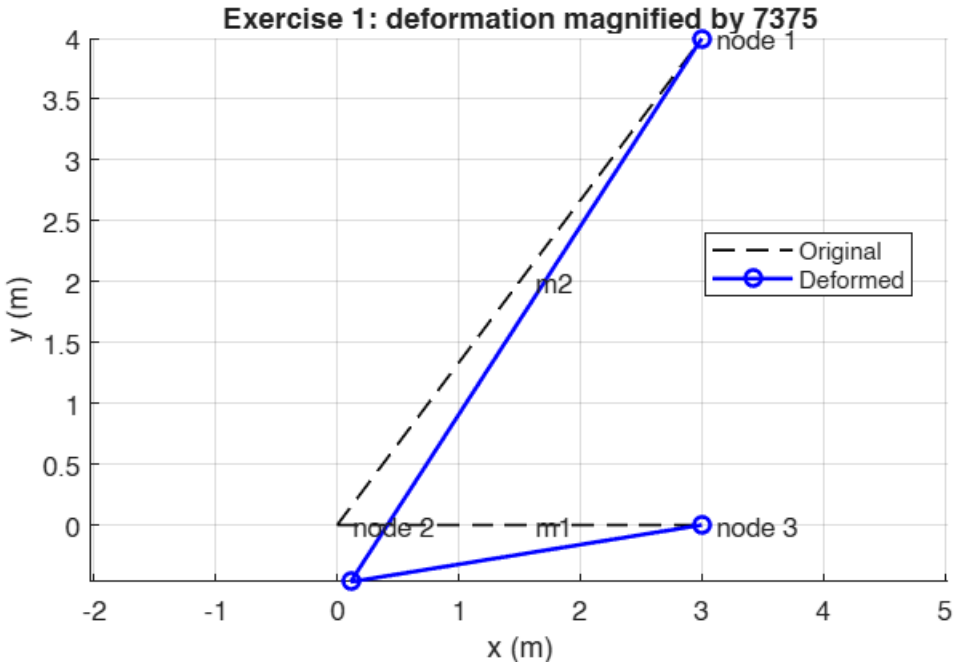
```
figure('Color','w');
hold on;
grid on;
axis equal;
```

`members(e,:)` selects the two nodes of member e. Plotting those two coordinate pairs draws the member. The format string `k--` means a black dashed line; `b-o` means a blue solid line with circular markers. The returned handles h0 and h1 identify plot objects for the legend.

```
for e = 1:nMembers
    ij = members(e,:);
    h0 = plot(nodes(ij,1),nodes(ij,2),'k--','LineWidth',1);
    h1 = plot(moved(ij,1),moved(ij,2),'b-o','LineWidth',1.5);
    midpoint = mean(nodes(ij,:),1);
    text(midpoint(1),midpoint(2),sprintf('  m%d',e));
end
```

`mean(...,1)` averages the two endpoint rows and produces the member midpoint [x y]. `text` places a label at a coordinate. `sprintf` builds a text string without printing it; `%d` inserts an integer member or node number.

```
for i = 1:nNodes
    text(nodes(i,1),nodes(i,2),sprintf('  node %d',i));
end
legend([h0 h1],{'Original','Deformed'},'Location','best');
xlabel('x (m)');
ylabel('y (m)');
title(sprintf('Exercise 1: deformation magnified by %.0f',scale));
```



The two handles passed to `legend` represent the two line styles, so the legend contains one entry for each configuration rather than one entry for every member. `%.0f` rounds only the displayed scale factor to a whole number. The geometry plot does not draw support or load symbols; the structural conditions are those defined earlier.